ABDULLAH GUL UNIVERSITY

Software Engineering Project Report

PAWFECT MATCH - ANIMAL SHELTER WEB APPLICATION

SUBMITTED BY

ÇAĞLA TOPRAK
MELDA PAKSOY
SEHER SESLİ
YUSUF AKTAŞ

MAY 2025

Content

LIST OF FIGURES

LIST OF TABLES

I. Project Description

1. Project Overview

The Animal Shelter Database Management System is a web-based software solution designed to streamline and enhance the process of animal adoption by connecting shelters and potential adopters through a centralized platform. This project aims to digitize shelter operations, enable the public to view and filter available animals, and manage visit scheduling effectively. It introduces a structured approach to animal information management, allowing shelter staff to update animal records, and enabling users to make informed adoption decisions based on reliable data and guidance.

By incorporating role-based access, visual content, filtering tools, and educational materials, the system serves as both an administrative tool and a community resource. Through this initiative, the project seeks to promote responsible pet ownership, increase adoption rates, and reduce the strain on shelter resources.

2. The Purpose of the Project

The rise in public interest in adopting pets, along with the growing need for shelter modernization, has created a strong demand for a robust, user-friendly digital platform. Many shelters still rely on manual records or fragmented systems, making it difficult to manage data efficiently or provide adopters with timely and accurate information. Our project acknowledges this gap and addresses the core challenges by developing a digital infrastructure that facilitates smooth communication and data sharing between shelters and the community.

2a. The User Business or Background of the Project Effort

Content

With the rise in pet adoption awareness and the increasing number of abandoned animals worldwide, animal shelters have become critical community resources. However, many shelters continue to operate using manual, outdated methods for managing animal data, adoption requests, and visitor flow. This leads to inefficiencies, data loss, and limited public accessibility to accurate animal information.

Our initiative aims to provide a centralized, digital platform that consolidates animal records, enables user-friendly adoption browsing, and allows potential adopters to make informed, responsible decisions. By offering detailed animal profiles, searchable filtering, and visit scheduling, the system creates an environment where users and shelters can engage transparently and efficiently.

Motivation

The driving force behind this project is the urgent need for a **modern, accessible solution** to bridge the communication and data management gap between animal shelters and the public. We believe that technology should serve both organizational efficiency and community education.

By creating a seamless digital experience, we aim to:

- Increase adoption success rates,
- Improve shelter operations,
- Support responsible pet ownership,
- And reduce shelter congestion through controlled visit scheduling.

Additionally, we recognize that adoption decisions are emotional and require trust. Our system builds that trust through transparency and detailed, reliable animal information.

Consideration

In designing this platform, we carefully considered the **diverse needs of both shelter employees and adopters**. On one hand, employees need an intuitive system to update animal records, manage visits, and communicate with adopters. On the other, users need visual, well-organized, and educational content to navigate adoption options effectively.

We also took into account:

- The need for mobile responsiveness,
- Secure user registration systems,
- Role-based access control,
- And features that encourage ethical adoption practices (e.g., feedback systems, breed-specific advice).

Recognizing the emotional and logistical significance of pet adoption, our platform is designed to make every stage—from discovery to decision—efficient, humane, and informed.

2b. Goals of the Project

Content

The primary goal of the Animal Shelter Database Management System project is to develop a comprehensive, user-oriented, and functional web platform that bridges the gap between animal shelters and potential adopters. The system is designed to serve two key user groups: shelter employees and general users seeking to adopt animals.

The platform allows shelters to:

- Digitally manage animal records,
- Upload detailed profiles and images of animals,
- Update information in real-time.

For users, it enables:

- Searching and filtering animals by species, breed, and other criteria,
- Viewing full animal profiles to aid in decision-making,
- Registering for accounts and scheduling shelter visits based on availability.

The integration of user accounts, profile management, and time slot-based visit scheduling ensures that adopters have access to structured and timely interaction with shelters, while shelters benefit from an organized system that limits overbooking and enhances resource allocation.

Motivation

Our motivation stems from the observed operational inefficiencies in many existing shelters, where adoption processes are often managed manually or with minimal technological support. These limitations result in a lack of transparency for adopters and an increased administrative burden on shelter employees.

By developing this platform, we aim to:

- Increase transparency and accessibility of animal information,
- Encourage responsible adoption through education and detailed data,
- Empower adopters to make informed decisions from the comfort of their homes,
- Enable shelters to manage their time and visitors more efficiently,
- Foster stronger human-animal connections by streamlining the adoption process.

This motivation is rooted in the belief that a well-designed, intuitive system can significantly enhance adoption success rates, reduce shelter overcrowding, and promote the overall well-being of animals.

Examples

Let’s walk through a practical example that reflects how the Animal Shelter Database Management System operates in a real-world scenario, strictly based on the features supported by the platform:

A registered user named Seher logs into the system with the intention of adopting a dog. On the homepage, she navigates to the "Explore Animals" section, where a list of animals currently available at the shelter is displayed.

Seher uses the built-in filtering tools and selects “Dog” from the species filter. The list instantly narrows down to show only dogs. Wanting to refine her search even further, she applies a gender filter and selects “Female.” Now, the system only displays female dogs.

From the remaining options, Seher begins reviewing the available dogs one by one. For each animal, she sees:

- A photo of the dog,
- The breed (e.g., Golden Retriever, Terrier, etc.),
- Age,
- Healthy information (chronic disease, temporary disease etc.),
- And the past information entered by the staff—such as whether the dog was previously adopted and returned, rescued from the street, or placed in foster care.

She comes across a female Golden Retriever named Luna, currently located in the park area of the shelter. Luna’s past information states that she was rescued from a suburban neighborhood and has stayed at the shelter for three weeks without any prior adoption attempts.

Satisfied with the details, Seher proceeds to schedule a visit. She opens the appointment booking module, selects Saturday at 2:00 PM, and initiates the adoption process. At that point, the system reserves that time slot for Luna.

On the shelter’s end, an employee named Can logs into the staff account and sees the new pending appointment. He checks the details and approves the visit with a single click. This approval is reflected on Seher’s user page as a confirmed appointment.

On the scheduled day, Seher visits the shelter, meets Luna, and decides to adopt her. Can completes the adoption process in the system by updating Luna’s status to “Adopted,” which automatically removes her profile from the public listings.

This scenario demonstrates how the system functions:

- It facilitates targeted animal searches using filters like species and gender,
- It presents staff-provided past information to support informed decisions,
- It organizes and automates visit scheduling for a smoother adoption flow,
- It allows shelter staff to efficiently track and manage appointments and adoption statuses, simplifying internal operations.

2c. Measurement

To ensure the effectiveness and value of the Animal Shelter Database Management System, we will employ a set of measurable performance indicators. These metrics will allow us to evaluate whether the project fulfills its intended goals—namely, improving the adoption process, supporting shelter operations, and enhancing user satisfaction. The data collected will also help guide future improvements and determine the overall success of the platform.

➢ User Engagement Metrics

- To assess the level of activity and participation on the platform, we will track:
- Number of registered users (both adopters and shelter employees),
- Frequency of logins and session durations,
- Volume of animal profile views and filtering interactions.

These engagement metrics will help us understand how actively the platform is being used and whether users are navigating toward meaningful adoption interactions.

➢ Visit Appointment Conversion Rate

One of the platform's key functions is visit scheduling. Therefore, we will measure:

- The number of scheduled visits compared to total adoption interest (e.g., animal profile views),
- The percentage of scheduled visits that result in actual shelter visits and completed adoptions.

This metric will allow us to evaluate how effectively the platform facilitates real-world outcomes, such as matching animals with responsible adopters.

➢ System Usage by Shelter Staff

To ensure the system provides value on the operational side, we will monitor:

- Number of animal entries created and updated by staff,
- Frequency of visit approvals and status changes (e.g., "Available" to "Adopted"),
- Time taken by staff to approve appointments after user submission.

These indicators reflect how well the system supports shelter workflows and whether it reduces administrative burdens.

➢ Data Accuracy and Record Completeness

We will evaluate the percentage of animal profiles that are fully completed with:

- At least one image,
- Breed and medical information,

- Location in shelter,
- Relevant past information (e.g., rescue background, foster history).

High completeness rates indicate the platform's success in enabling transparent and informed adoption decisions.

➢ Platform Accessibility and Responsiveness

To assess usability, we will monitor:

- Page load times for major sections (Explore Animals, Profile Pages, Appointment Module),
- Error rates or failed form submissions,
- Mobile responsiveness and browser compatibility testing results.

Improved accessibility ensures that both users and shelter staff can interact with the system reliably across various devices.

➢ Adoption Status Tracking

Lastly, the adoption flow will be measured by:

- Number of animals successfully marked as "Adopted",
- Time elapsed from first profile view to status change,
- Frequency of status updates by staff.

This will reflect the system's ability to guide users through to successful, documented adoptions.

By implementing and analyzing these measurement strategies, we will be able to determine how well the system addresses the real needs of both adopters and shelter staff. These metrics not only validate the system's performance but also provide valuable insights for iterative development, ensuring long-term success and sustainability of the platform.

3. The Scope of the Work

3a. The Current Situation

Animal shelters often struggle with fragmented or outdated systems when it comes to managing animal data and coordinating adoption processes. In many cases, records are maintained manually or through basic digital tools that lack integration, standardization, and real-time accessibility. This makes it difficult to provide accurate, comprehensive, and timely information to potential adopters, and also burdens shelter staff with repetitive administrative tasks.

There is currently a noticeable absence of an all-in-one digital platform tailored specifically for shelter needs—one that can handle animal information, visit appointments, and user interactions from a single interface. Our project directly addresses this issue by introducing a structured web-based solution where all these elements are brought together. The system enables shelter personnel to manage records efficiently, while allowing users to explore adoptable animals, view detailed profiles, and book visit appointments through an intuitive interface.

By creating a centralized and easy-to-use platform, the project aims to replace disconnected processes with a streamlined adoption workflow that benefits both the shelter and the community.

3b. The Context of the Work

Public interest in animal adoption has grown significantly in recent years, alongside an increasing demand for transparency, accessibility, and humane treatment within animal welfare systems. At the same time, animal shelters continue to operate under tight budgets, limited staff, and rising animal intake numbers. These conditions call for a scalable and intelligent solution that can help shelters operate more effectively while also improving the adoption experience for users.

Within this broader social and operational landscape, the **Animal Shelter Database Management System** is designed to offer more than just a digital record-keeping tool—it becomes a bridge between shelter operations and public engagement. The system introduces a balanced digital ecosystem where:

- Shelters gain control over their internal processes through structured data entry, status tracking, and appointment management,
- Prospective adopters gain access to clear, updated, and trustworthy information that empowers them to make thoughtful decisions.

This project is developed in an academic setting but modeled to reflect real-world usability. By aligning itself with current adoption trends and operational challenges, it aims to become a prototype for how technology can elevate the quality, speed, and ethics of the animal adoption process.

4. Product Scenarios

Product scenarios are informal yet detailed narratives that describe how different types of end users will interact with the Pawfect Match Animal Shelter system once it is fully developed. These scenarios demonstrate the system's core functionalities in realistic use cases, often involving specific user roles such as shelter staff and potential adopters. The purpose of these scenarios is to illustrate how the platform will support real-life workflows and solve practical problems within the adoption process.

4a. Product Scenario List

ANIMAL FILTERING	USER INTERACTION	DATABASE (MYSQL)
• Filter by species • Filter by gender • Filter by age • Filter by neutered status • Filter by chronic disease status	• View animal profile and information • Get an appointment • Adopt the animal	• Store user information • Store animal data • Update a adoption status • Track appointment logs
STAFF ACTIONS	**NOTIFICATION SYSTEMS**	**LOGIN & SECURITY**
• Add new animal • Approve and reject adoption • Add new article and blog	• Notification of approval after the ownership process • Confirmation pop-up after adding an animal	• User/admin login • Role-based access control • Change password with SHA-256 algorithm

Figure 1. Product Scenario List

4b. Individual Product Scenarios

Scenario 1: Browsing and Filtering Animals

Seher, the registered user, logs into the Pawfect Match Animal Shelter website. On the homepage, she navigates to the "Explore Animals" section, where she is presented with a list of animals currently available for adoption.

Using the filtering options, Seher selects "Dog" under species and then applies a gender filter for "Female." She also filters by age to see only young animals. The system quickly updates the list to show only female dogs that meet her criteria.

She clicks on individual profiles to view details such as photos, breed, medical status, and past information entered by shelter staff. This allows Seher to narrow down her choices confidently.

Scenario 2: Scheduling a Visit

After selecting a dog named Luna, Seher decides to schedule a visit. She opens the appointment module, selects an available date and time, and submits her request.

The system blocks the selected time slot for Luna and sends a notification to the shelter staff. An employee, Can, reviews the request and approves it with one click. Seher receives a confirmation email and sees the approved appointment in her dashboard.

Scenario 3: Adoption Confirmation and Status Update

On the scheduled day, Seher visits the shelter, meets Luna, and chooses to adopt her. Can update Luna's status in the system to "Adopted." This change removes Luna from the public listings and archives her profile for record-keeping.

Seher is sent a notification confirming the completion of the adoption process, and she gains access to post-adoption resources via her account.

Scenario 4: Staff Adds a New Animal Profile

A shelter employee logs into the admin panel to add a newly rescued animal. The employee fills out a form including species, gender, age, medical status, location within the shelter, and any past information.

Once submitted, the animal profile appears immediately in the public listing for adopters to view and filter.

5. Stakeholders

5a. The Client

The client serves as the principal stakeholder responsible for the strategic direction, resource allocation, and oversight of the Pawfect Match Animal Shelter project. Their role encompasses articulating clear project objectives, establishing detailed requirements, and ensuring alignment with organizational goals related to animal welfare and shelter management. Throughout the development lifecycle, the client maintains active collaboration with the project team, providing critical feedback and making informed decisions that shape the platform's functionality, usability, and compliance with legal and ethical standards. The client also facilitates stakeholder engagement by coordinating with shelter authorities and supporting dissemination and adoption of the system post-deployment.

5b. The Customer

The customer base for the Pawfect Match Animal Shelter system consists primarily of two distinct groups, each with unique needs and interaction modalities:

- **Shelter Staff:** This group includes animal care professionals, administrative personnel, and volunteer coordinators who require efficient digital tools for managing comprehensive animal records, scheduling and approving visitations, and maintaining up-to-date adoption statuses. Their use of the system is critical to ensure operational accuracy, data integrity, and regulatory compliance within shelter activities.

- **Prospective Adopters:** Individuals, families, or organizations seeking to adopt animals represent the second primary customer group. Their interaction focuses on exploring the available animal database, utilizing search and filter functions,

accessing transparent animal histories, and coordinating shelter visits. This group values intuitive interfaces, reliable information, and smooth communication channels that facilitate confident decision-making.

The system is designed to mediate and balance these often divergent requirements, fostering efficient workflows and enriching the adoption experience.

5c. Hands-On Users of the Product

The hands-on users of the Pawfect Match platform encompass a broad spectrum of profiles, each engaging with the system in distinct capacities:

- **Shelter Personnel:** They perform daily operational tasks such as animal profile creation and updates, appointment management, and adoption status tracking. Their role demands reliable, fast, and secure access to backend functionalities.
- **Adopters and Visitors:** Registered adopters interact primarily with the frontend interfaces to search, filter, review animal profiles, and schedule visits. They require intuitive navigation, comprehensive yet digestible information, and prompt feedback mechanisms.
- **Volunteers and External Partners:** Although indirect users, volunteers and partner organizations utilize the platform for coordination and data sharing, influencing broader ecosystem effectiveness.

The platform's design emphasizes accessibility and usability, ensuring all users, irrespective of technical expertise or background, can perform their respective roles efficiently.

5d. Priorities Assigned to Users

User priorities within Pawfect Match have been carefully stratified to optimize resource allocation and feature development:

- **Primary Users:** Shelter staff and adopters hold the highest priority as their needs directly impact the core functionality and success of the platform. Enhancements focus on streamlining workflows, minimizing friction in adoption processes, and maximizing user satisfaction through responsive interfaces and reliable data.

- **Secondary Users:** These include volunteers, animal welfare advocates, and casual visitors who engage less intensively but contribute to the ecosystem through support activities and awareness efforts. The platform accommodates their needs by providing informational resources and limited interaction capabilities.

- **New Users:** Onboarding new adopters and staff with comprehensive tutorials, guided tours, and easy-to-understand documentation is prioritized to reduce barriers to entry and promote sustained engagement.

This tiered prioritization ensures balanced attention across stakeholder groups, facilitating a cohesive and scalable platform environment.

6. Mandated Constraints

6a. Solution Constraints

Content

Description: Only registered users (shelter staff and adopters) can access the Pawfect Match Animal Shelter platform. Users are required to log in to the system and role-based access control is applied for different roles. The system restricts data access and transaction rights of users according to their authorisation level. For example, only shelter staff can add and edit animal information, while adopters can only view animal profiles and book visits.

Due to security and personalisation requirements, the system requires strong password policies, multi-factor authentication options (in future versions) and encrypted communication with HTTPS protocol. Furthermore, backup, fault tolerance and load balancing mechanisms must be integrated in the database and application servers to ensure that the system meets the criteria of high availability and scalability.

Rationale: These restrictions are necessary to ensure that the platform is secure, sustainable and user-friendly. Access restriction through registered users prevents malicious access and protects data confidentiality. Role-based access guarantees that transactions are carried out by the right authorities, while security protocols ensure that user information is protected.

Fit Criterion: The platform should have a strong and reliable login system that ensures that only authorised users can access features. Furthermore, different levels of authorisation should be supported, distinguishing between user roles.

6b. Implementation Environment of the Current System

Content

Pawfect Match Animal Shelter project is developed using Microsoft's ASP.NET MVC framework. C# language is preferred on the backend side and database operations are managed with Entity Framework ORM tool. HTML and CSS, which are basic web technologies, were used in the frontend design. This technological structure makes the application modular, scalable and sustainable.

MySQL is preferred for database management and Entity Framework provides performance and reliability by facilitating data access between the database and the application. The ASP.NET MVC architecture organises the application's business logic, user interface and data access in a layered and decoupled manner, which facilitates maintenance, testing and extensibility.

The application is optimised to run in a Windows Server environment and is hosted on IIS (Internet Information Services). The frontend side is structured according to responsive design principles to enhance the user experience and works in accordance with different screen sizes.

Motivation

The combination of ASP.NET MVC and Entity Framework is favoured for developing a robust and secure web application. This technology stack offers the advantages of strong type checking, ease of code reuse and integration in the Microsoft ecosystem. It also accelerates the management and development of workflows by providing clear separation between backend and frontend.

The use of Windows Server and IIS is appropriate to meet the high availability and performance requirements of the application. The responsive frontend design guarantees that the platform is user-friendly on both desktop and mobile devices.

6c. Partner or Collaborative Applications

Content

This section defines external or in-house applications that are not direct components of the Pawfect Match Animal Shelter system but are in integration or cooperation with the product. In our project, backend development is carried out using ASP.NET MVC framework and database management is carried out on MySQL.

ASP.NET MVC offers a layered and modular structure with Model-View-Controller pattern in system architecture, while MySQL database provides high performance, relational data management. There is a tight integration between these two main technologies; ASP.NET application optimized data access by connecting to MySQL database with ORM layers such as Entity Framework.

The project has not yet integrated with third-party applications such as e-mail services, instant messaging platforms, external APIs or commercial software. However, the current architecture has the flexibility to seamlessly integrate with such partner applications in the future.

Motivation

The use of partner applications can impose significant constraints and requirements on the design and development process. In the current ASP.NET MVC and MySQL based architecture, data exchange protocols, performance optimisations and error management between these two systems are critical. In case of integration with partner applications, additional design parameters such as API compatibility, security protocols and data consistency come into play.

Therefore, a clear definition of ASP.NET MVC and MySQL integration in the existing structure is necessary to increase the durability and sustainability of the system. In addition, compatibility and data synchronisation issues that may arise in the integration processes of partner applications that may be added in the future should be analysed in advance.

Examples

In the Pawfect Match platform, the ASP.NET MVC application interacts with the MySQL database through the Entity Framework. For example, the registration of visit appointments and approval processes are provided by coordination between MVC Controllers and MySQL tables. Furthermore, animal information and user registrations are processed into the database through secure CRUD operations.

In the future, for example, Twilio API for SMS notifications or SendGrid integrations for e-mail services can be planned; RESTful API interfaces and OAuth-based authentication systems can be designed for such partner applications.

Consideration

While developing the system, the performance limits, security vulnerabilities and scalability needs of the integration between ASP.NET MVC and MySQL should be meticulously analysed. In partner application integrations, data consistency and latency will be critical evaluation criteria.

When modelling workflows, the API capabilities, data formats and security standards of the applications to be collaborated with should be taken into consideration. In addition, it should be ensured that the existing system architecture has the flexibility and expansion capacity to handle these new integrations.

Although the Pawfect Match project was primarily built on the existing ASP.NET MVC-MySQL infrastructure, the software architecture was kept modular and open, thus creating the possibility of seamless integration with different partner applications in the future.

6d. Off-the-Shelf Software

Content

In the development of the Pawfect Match Animal Shelter platform, various off-the-shelf software components and open source libraries were used to provide basic functionality quickly and reliably. For the frontend design and user experience, HTML and CSS as well as popular open source CSS frameworks such as Bootstrap were preferred. In backend development, Microsoft's ASP.NET MVC framework was used and Entity Framework ORM tool was integrated for data access and management.

In database management, a widely used relational database system such as MySQL supports the basic data operations of the project thanks to its ready and mature structure. In addition, commercial development tools such as Visual Studio IDE play an active role in coding, debugging and performance analysis.

Motivation

The use of off-the-shelf software components speeds up the development process, allowing the project to be completed on time. At the same time, these components reduce costs and provide high quality, standards-compliant solutions. Open source frameworks and libraries are constantly updated with broad community support and are supported by security patches. This contributes positively to the sustainability and flexibility of the platform.

Ready-made ORM solutions such as Entity Framework increase productivity by enabling developers to manage database operations more easily and error-free. UI frameworks such as Bootstrap facilitate the creation of consistent, accessible and responsive designs.

Consideration

The versions and compatibility of the off-the-shelf components used should be continuously monitored, software updates and security patches should be applied regularly. Integration compatibility and behavioural differences between components should be carefully analysed and situations that will adversely affect system performance should be identified in advance.

In addition, the licence conditions of open source and commercial components used in the project should be examined and legal compliance should be ensured. When necessary, customisation of components or evaluation of alternative software may be critical for the long-term success of the project.

6e. Budget Constraints

Content

The budget allocated for the Pawfect Match Animal Shelter project is a critical element that must be carefully managed at every stage of the software development process. The budget covers basic items such as software development, testing, server infrastructure, maintenance and support services. In addition, licence fees, use of third party components and possible outsourcing services are also included in the budget planning.

In order to reduce costs and use resources efficiently, open source technologies (e.g. MySQL, Bootstrap) and existing in-house infrastructures were prioritised. Budget constraints necessitate feature prioritisation during the development process, preventing unnecessary expenditures and allocating resources in accordance with the project objectives.

Motivation

Compliance with budget discipline is fundamental to the economic sustainability of the project. Effective management of resources ensures that critical functions are fully developed and that the system meets user requirements. In addition, budget planning supports adherence to the project schedule and contributes to on-time delivery.

Careful budget management enables the development of a high quality and reliable platform that meets the expectations of users. Thus, the return on investment is maximised, ensuring the long-term sustainability of the project.

7. Naming Conventions and Definitions

Adopter : Refers to individuals or families who want to adopt an animal on the platform. These users can filter the animals, get detailed information, plan a visit and start the adoption process.

Shelter Staff: Shelter staff responsible for managing animal records, approving visits and updating adoption status. These users have special authorisations on the platform.

Animal Profile: It is a digital record where detailed information (species, gender, age, health status, past information, photos, etc.) of each animal in the shelter is collected.

Visit Appointment: These are the bookings created by the adopters through the system in order to plan their visits to the shelter. These appointments are confirmed and managed by shelter staff.

Adoption Status : Indicates the current status of an animal in the adoption process. For example; It includes statuses such as ‘Available’, 'Reserved”, ‘Adopted’.

These definitions ensure that the key terms used in the Pawfect Match platform are clear and understandable. This creates a common language and understanding between all stakeholders.

8. Relevant Facts and Assumptions

8a. Facts

- The Pawfect Match Animal Shelter project was developed using the ASP.NET MVC framework; this technology provided a familiar and powerful infrastructure for our team.
- MySQL was chosen as the database of choice, allowing data to be stored securely, consistently and accessible.
- The project targeted two main user groups: shelter staff and adopters.
- The platform allows users to filter and access detailed information on animals, schedule visit appointments and manage the adoption process.
- During the development process, roles were clearly distributed within the team and regular communication and co-operation were ensured.

- The system has been optimised to work smoothly on desktop and mobile devices in line with responsive design principles.

8b. Assumptions

- It is assumed that the users have basic knowledge of computer and internet usage; thus, the user interface of the platform is designed in a simple and understandable way.
- It is assumed that shelter staff will actively use the platform and update animal data regularly.
- It is assumed that tracking visit and adoption appointments through the system will facilitate in-shelter operations and increase user satisfaction.
- Although there are currently no e-mail or SMS notification systems, it is assumed that these features can be added at later stages.
- It is accepted that the infrastructure of the platform should be scalable depending on the increase in the number of users.

II. Project Issues

9. Open Issues

A detailed assessment has not yet been made in terms of legal compliance on how users' personal data will be protected and which data will be stored for how long.

Due to differences in the system infrastructures of shelters, our system may not work smoothly in every shelter.

The system's currency depends on shelter staff regularly entering information. If regular information is not entered, the system may lose its currency.

10. Off-the-Shelf Solutions

10a. Reusable Components

To make this project a secure, maintainable, and scalable web application, we leveraged several key reusable components and frameworks. Some of these components are:

ASP.NET Core with MVC Architecture:
We used ASP.NET for the back end of our project. We used the MVC (Model-View-Controller) architecture to facilitate the development, testing and maintenance of the application by providing a clear separation of concerns.

Entity Framework Core:
EF Core is an Object-Relational Mapping (ORM) tool for data access. We used this tool to simplify communication between the platform and the MySQL database and to be able to work with database records using .NET objects.

MySQL Database:
MySQL is a widely used open source relational database system. In the project, we used the MYSQL database system to create and store animal information, user profiles, visit plans and adoption records because it provides scalable and reliable data storage.

11. New Problems

11a. Effects on the Current Environment

The PawfectMatch platform will affect the current environment significantly. Current animal shelters are using disconnected and mostly non-digital tools in shelter management. With the usage of our system shelter managements will be more efficient, interconnected and paperworks will be reduced.

Expected Effects:

1. Interconnected Shelter Data: All data about shelter will be stored in a database securely. This will reduce the risk of data loss and inconsistency and increase the dependency over data.
2. Automated Adoption Process: Human error and staff workload will be decreased by semi-automating manual operations like scheduling appointments and matching animals to potential adopters. At the same time, adoption processes will be more trackable.
3. Accessibility: By using web browsers, staff members will be able to view and edit information remotely, fostering current workflow and operational flexibility additionally people can be informed about animals in shelters.

11b. Effects on the Installed Systems

The large-scale corporate systems that shelters presently employ cannot be replaced directly by the PawfectMatch, because it is intended to be a stand-alone web service. However, manual data conversion or data import systems can be replaced like digital forms and spreadsheets.

11c. Potential User Problems

Despite PawfectMatch has many advantages, there are some potential user problems:

1. Employees in shelters could find the system difficult.
2. Dependency on the internet could be problematic in places with unreliable connectivity such as shelters in rural areas.
3. Users who are not adapted to digital technologies may encounter challenges.

11d. Limitations in the Anticipated Implementation Environment That May Inhibit the New Product

There are a few limitations in the anticipated implementation environment that may inhibit the new product:

1. Technical Constraints: Some shelters may not have enough technologic components like stable internet access, insufficient computers or staff without technical training.
2. Resistance of Shelter Staff: Changes in workflows can be an unwanted thing among shelter staff who prefer more traditional methods.

3. Guideline Differences: The system might not work completely across all shelters if adoption procedures or data-sharing guidelines are not uniform among shelters.

4. Security Concerns: Concerns about data privacy and cyberattacks may make some shelters hesitant to use a centralized online system

12. Tasks

12a. Project Planning

The development of the PawfectMatch platform follows Scrum Method which is an Agile approach with iterative feedback and adaptation. This approach ensures customer feedback, flexibility and clear documentation.

12b. Planning of the Development Phases

The entire procedure was broken down into manageable tasks with clear deliverables during the planning stages of the development phases. Each step built on the previous one to design an extensible reliable platform

Main planning phases include:

Phase 1: Requirements Analysis

Description: The main user roles and basic functions were defined. Based on these, it was decided that the project would be a web project.

Date: 18/02/2025

Phase 2: System and Database Design

Description: ASP.NET with MVC framework is decided to use in backend development because of its useful architechture. Appropriate databases were identified for the project and MySQL was selected due to its suitability for the course and project requirements.

Date: 20/02/2025

Phase 3: Starting to Backend Development and Frontend

Description: After the first project meeting we started to backend and frontend development to complete tasks. We made a simple useful UI design.

Date: 04/03/2025

Phase 4: Testing and Debugging

Description: Testing the platform by using sample cases. Debugging with gray box, white box and black box debugging methods.

Date: 10/04/2025

Phase 4: Code Reviewing

Description: After the completion of tasks for each iteration codes are reviewed.

Date: 12/04/2025

Phase 5: The Last Project Meeting

Description: Finalizing all tasks in iterations. Taking last customer reviews.

Date: 06/05/2025

Phase 6: Finalizing the Project

Description: Completing last tasks from user review. Launching the first demo. Presenting the project.

Date: 13/05/2025

13. Migration to the New Product

13a. Data That Has to Be Modified or Translated for the New System

To ensure a smooth and complete transfer, a number of critical data-related requirements must be met when migrating to the PawfectMatch platform. These features focus on maintaining data integrity during the conversion process, preparing and converting historical data for the new environment.
The primary tasks to consider during this migration phase include:

1. **Data Migration and Translation:**
   Current animal shelters may use disconnected forms, documents and non-digitized tools. Before the usage of PawfectMatch, these data should be collected, joined and migrated to the PawfectMatch's database.

2. **Data Compatibility:**
   Our terminology and value types in the platform should match with the current shelter. Such as breeds or disease terminologies or date types.

3. **Training Shelter Staff:**
   Some staff in shelters may not have enough experience with digital technologies or they can struggle with adapting our system. In this case training materials for shelter staff should be created.

14. Risks

Every project can involve risks. Despite careful preparation, PawfectMatch can still face a number of challenges due to changing requirements, technical uncertainties, and human issues. Some of the imported risks are mentioned in this section.

Possible risks for PawfectMatch:

1. **Incomplete or Inconsistent Data** (Probability: Medium)
   The entered values can be missing or erroneous. This type of data can cause errors in processes. We use constraints in the database to minimize this risk.

2. **Poor User Adoption or Resistance** (Probability: Medium)
   Another important risk is that users may resist or fail to adapt to the new system. Manually working shelter workers may have difficulty switching to a digital platform. In order to minimize this risk, an intuitive user interface was developed in our project. In addition, basic usage scenarios can be created in the future and training and guide content can be prepared for the system.

3. **Security Vulnerabilities** (Probability: Medium)
   Security vulnerabilities are a major problem, especially in systems where user data is located. In the PawfectMatch project, precautions such as using hashed user passwords have been taken. In addition, regular tests and code reviews can be performed against such risks.

4. **Data Loss During Migration** (Probability: High)
   During data migration, there is also a significant risk of data loss and technological errors. To minimize this risk, data from the previous system can be backed up, migration procedures can be tested in test environments, and backup plans can be made in case something goes wrong.

5. **Low Performance or Scalability Issues** (Probability: Low)
   Although risks such as performance issues or the system's inability to accommodate increasing numbers of users are currently considered low-probability risks, this risk can be further reduced by strengthening the system and optimizing database queries.

15. Costs

The cost assessments for creating and implementing the PawfectMatch platform include a number of elements related to the technological infrastructure and time-based effort of the development team. This section will address the potential costs of fully implementing the system.

Domain Registration:
A domain name must be registered in order for the PawfectMatch website to be available. The domain extension (.com, .org, etc.) and the domain name registrar chosen can affect the price. Domain name registration costs typically range from $10 to $50 per year, depending on the popularity and availability of the desired name.

Hosting Services:
Reliable hosting is essential for users to access the system without any issues. Hosting type can be shared, VPS, or cloud-based. Expected traffic and storage needs affect hosting prices. Cloud-based or scalable VPS options can cost between $50 and $100 per month, depending on performance and uptime requirements.

SSL Service:
An SSL certificate is required to protect data sent between users and the platform. Depending on the certificate type and provider, solutions with higher validation levels can cost anywhere from $10 to $200 per year, while basic SSL certificates can be purchased for free.

Website Development and Design:
The cost of creating a platform that uses ASP.NET Core MVC, Entity Framework Core, MySQL and Bootstrap tools in its development can vary from a few hundred dollars to several thousand dollars or more.

Maintenance, Updates and Support:
Ongoing maintenance is required after launch to keep the system secure, up-to-date, and error-free. This includes patching vulnerabilities, managing backups, and server software and security updates. Monthly maintenance packages typically cost between $20 and $200, depending on the frequency of updates and the degree of technical support required.

Buna göre, PawfectMatch tam ölçekli bir ürün olarak uygulanacak olsaydı, ilk kurulum masrafları (alan adı, barındırma, SSL ve geliştirme) 2.500 ila 6.000 dolar arasında olabilir ve ölçeklenebilirliğe ve özellik artışına bağlı olarak yıllık bakım maliyetleri zamanla 2.000 dolara kadar çıkabilirdi.

16. Waiting Room

Since the time we spent developing the PawfectMatch platform was limited, we focused more on core functions during this process. But we designed PawfectMatch as an extensible platform therefore there are some ideas in our waiting room. In this section, we list some of these features.

1. **Push Notifications / Email Alerts**
   Actions like approved or rejected adoption requests can be sent to users as a notification via email.

2. **Lost Pet Finding System**
   Users can send the information of their lost pet like photo, health information, characteristic features and the system can detect that animal if it came to shelter. Also other users can see this information and contact the adopter if they think they found that animal.

3. **Multi-Language Support**
   Now our project supports just English but we can add extra languages.

4. **Donation System**
   There can be a donation page showing the shelter requirements. Via this window users can choose a requirement and donate for it.

5. **Chat Between Shelter and Adopter**
   There can be a chat window to enable extra contact among shelter staff and potential adopters.

6. **Adoption Recommendation System**

   In the current system, users are finding the animals which they want to adopt manually. But there is an option of an automated recommendation system which is selecting animals fitting with users' conditions.

17. Ideas for Solutions

We have some solutions for ideas mentioned in section 16. This section defines these solutions:

1. Adding a credit card option for donations.
2. Using an AI for adoption recommendation system.
3. Using an LLM and image processing system for lost pet function.

18. Project Retrospective

It is important to reflect on the factors that helped the project succeed and the challenges encountered throughout the process. This section aims to identify areas that require change or improvement and successful techniques that should be repeated in subsequent projects.

Successful Techniques:
Using the MVC framework helped us to work in a team thanks to its discrete achitechture. After each project meeting we clarify the tasks and divide it among our team clearly. This helped us to complete all requirements in iterations easily.
Using Entity Framework ensured easy integration of database operations with object-oriented programming.

Unsuccessful Techniques:
Adding design patterns to code later caused time loss.

III. Design

19. System Design

19a. Design Goals

Correctness
The system must display accurate and up-to-date information about animals available for adoption. Shelter employees should be able to input, update, and manage data without introducing inconsistencies. Users should also receive reliable feedback and confirmation when submitting adoption requests.

Robustness
The application is designed to handle incorrect or unexpected user inputs gracefully. Form validations are implemented to prevent missing or invalid data, especially during registration, login, animal entry, and adoption request submission.

Security
Since the system includes login functionality for both shelter employees and users, data security is a critical goal. Sensitive operations like user authentication, animal management, and adoption approval are protected against unauthorized access.

Scalability and Flexibility
The software architecture is built in a modular fashion, allowing for future expansion such as integrating additional features like donation tracking, chat with shelters, or animal medical history. The system should also be adaptable for use by different shelters.

Efficiency
The system uses efficient database queries to ensure that filtering animals and processing adoption requests is fast and responsive. Page load times and server interactions are optimized to provide a smooth user experience.

User-Friendliness
The platform aims to be intuitive and easy to use for both shelter employees and general users. It includes a clear layout, helpful filters for animal search, and simple navigation. Users with minimal technical background should still be able to successfully complete an adoption process.

19b. Current Architecture

A clean and modular architecture is essential for collaborative development and long-term maintainability. In our web application project, we structured the software using the ASP.NET MVC (Model-View-Controller) pattern, which allows us to separate the application logic, data, and user interface components effectively.

1. Presentation Layer (Views and Controllers)

This layer handles the interaction with users. Views are responsible for rendering the user interface, and Controllers manage the flow of data between the views and the model.

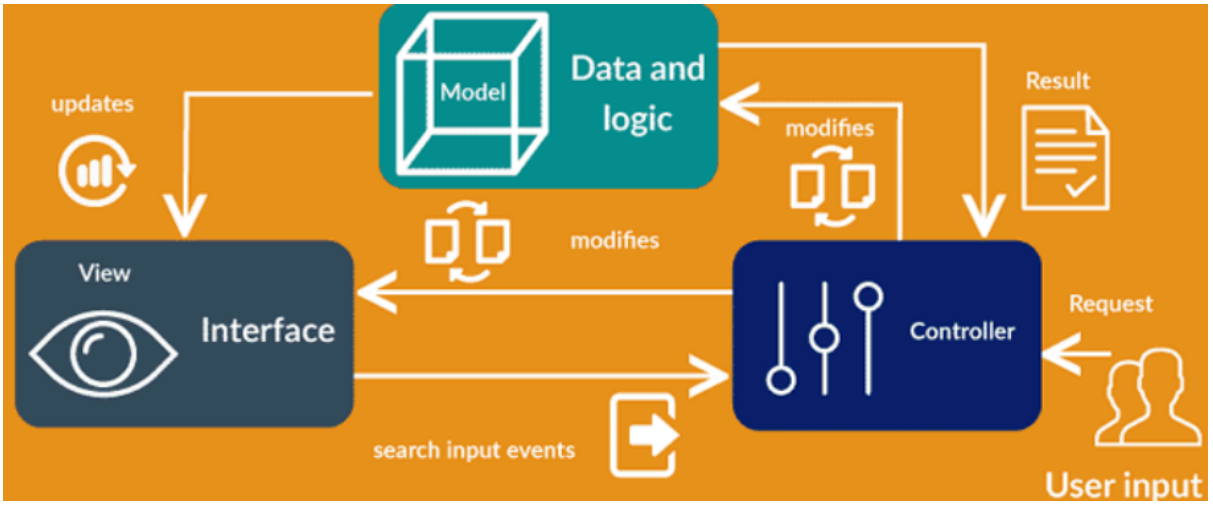

Figure 2. MVC Architecture

2. Business Logic Layer (Controllers and Services)

In this layer, we process user requests, apply business rules (e.g., adoption conditions), and manage workflows. Controllers also validate user inputs and handle adoption requests.

3. Data Access Layer (Models & MySQL)

This layer is responsible for interacting with the database. We used MySQL as our database system. Entity classes represent the tables in the database and are used for CRUD operations.

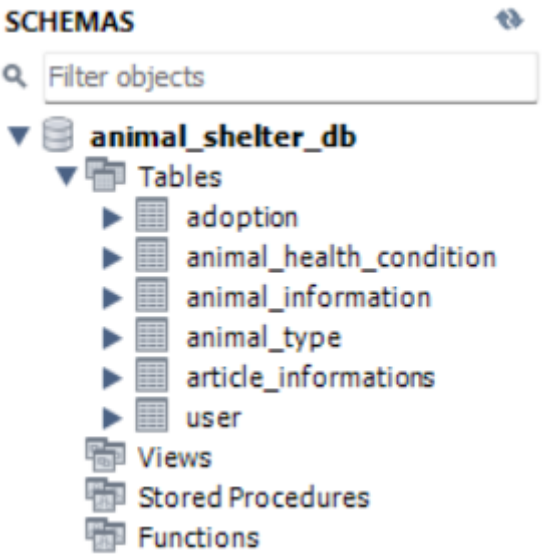

Figure 3. Database schemas

4. Use Case Layer

This refers to the logical grouping of features based on different user roles. In our system, there are two main roles: Shelter Employees and Registered Users. Each role has specific interactions with the system (e.g., employees can add/update animals; users can view and request adoptions).

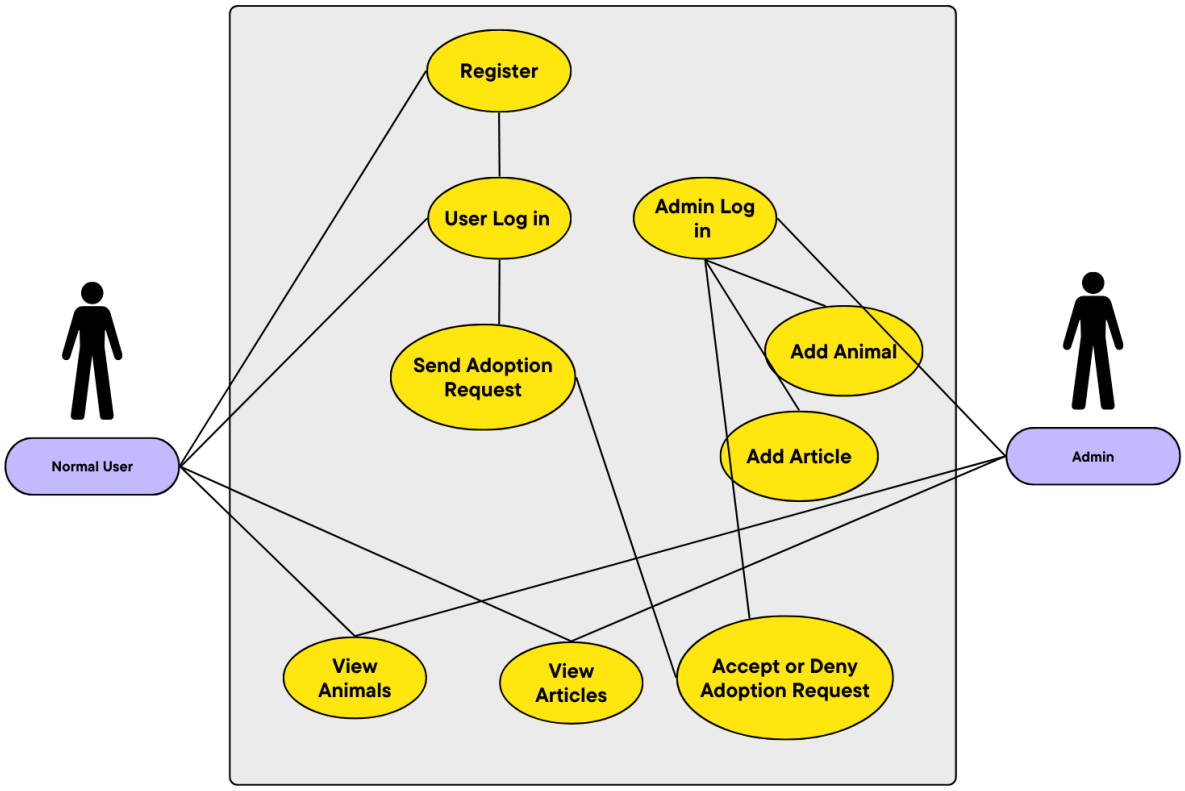

Figure 4. UML Diagram

5. Framework & Technologies

We used ASP.NET MVC Framework for developing the web application. It follows a layered approach and supports rapid development with strong separation of concerns. Our system runs on the .NET runtime and communicates with MySQL via an ORM or connector.

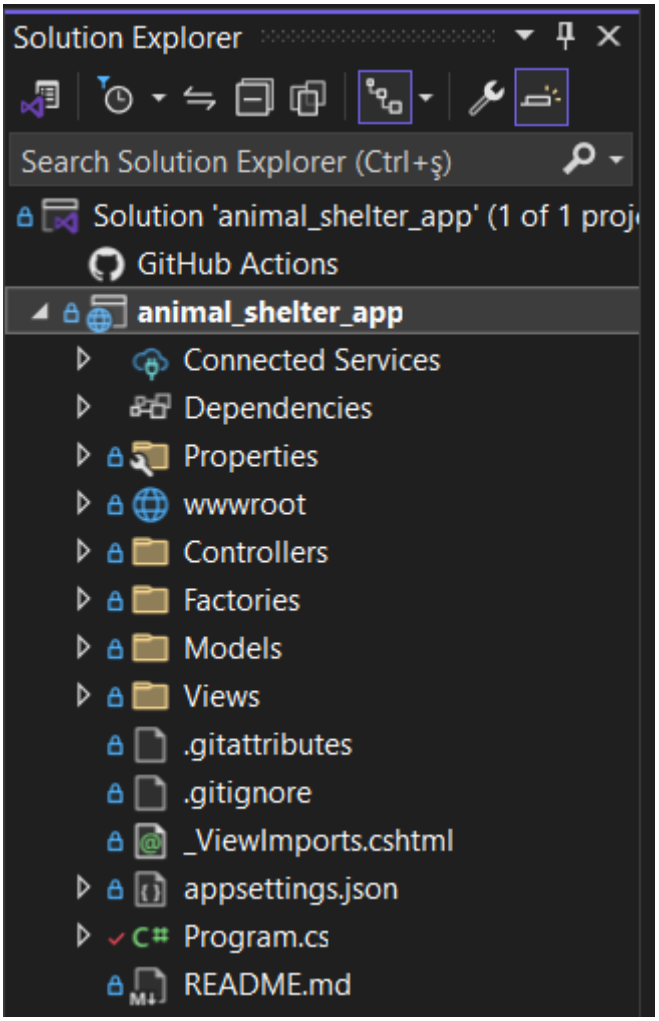

Figure 5. Project Directory Structure

19c. Proposed Software Architecture

Overview

The proposed architecture of our web-based animal adoption system follows the Model-View-Controller (MVC) pattern provided by the ASP.NET framework. This design enables a clear separation between data, application logic, and user interface. It improves maintainability, scalability, and ease of collaboration among developers.

The system is layered as follows:

- Presentation Layer: Handles user interface and browser interactions (Views)

- Controller Layer: Contains application logic and handles request processing
- Model Layer: Manages data and communicates with the database using entity classes

Class Diagrams

- The system's class diagram illustrates the core entities, their attributes, and relationships. It includes main entities such as User, Animal, Adoption, and related structures for animal health, type, and articles.
- Each class corresponds to a database table and is implemented as a model in the MVC structure. Relationships are modeled via foreign keys and are reflected in class associations.

Dynamic Model

The dynamic model describes the behavior of the system during runtime, especially focusing on user interactions such as adoption requests.

Example Sequence:

1. A registered user views available animals.
2. The user submits an adoption request.
3. The system sends this request to the admin dashboard.
4. Admin approves or rejects the request.
5. Status is updated in the database, and the user is notified.

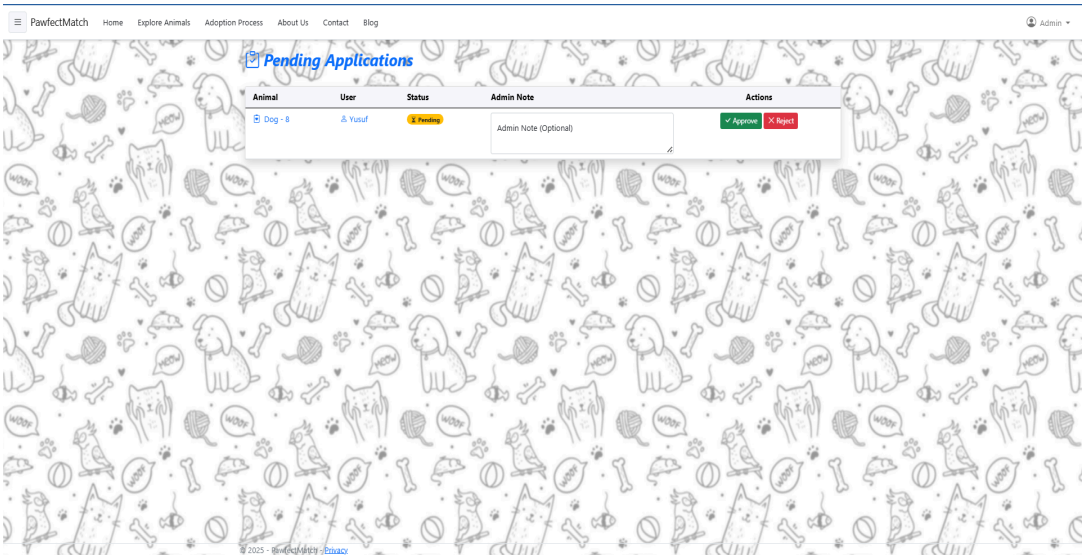

Figure 6. Adoption request shown in admin dashboard

Subsystem Decomposition

The system is logically divided into the following subsystems:

- User Management Subsystem: Handles registration, login, and user role distinction.
- Animal Information Subsystem: Stores, updates, and displays animal details.
- Adoption Request Subsystem: Manages request submission, approval, and status tracking.
- Blog/Article Subsystem: Delivers informative content to users.

Hardware / Software Mapping

- Hardware Requirements:
    - Client: Any modern web browser (desktop or mobile)
    - Server: IIS-enabled Windows Server with .NET runtime
    - Database Server: MySQL server installation
- Software Components:

- Frontend & Backend: ASP.NET MVC (C#)
- Database: MySQL
- Development Environment: Visual Studio, MySQL Workbench

Data Dictionary

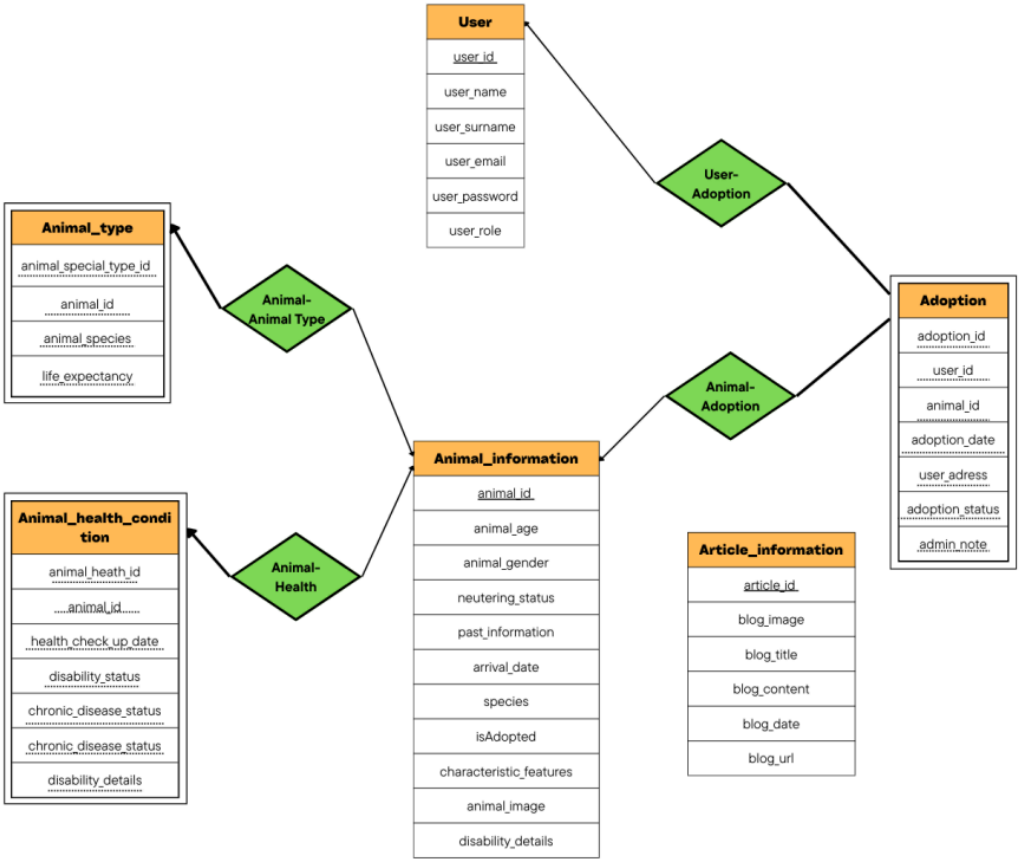

Figure 7. ER Diagram

Persistent Data Management

Data persistence is handled using MySQL, where each core entity is represented as a table. CRUD operations are performed via ASP.NET MVC controllers using ORM or direct SQL queries.
Adoption requests, user accounts, and animal data are saved persistently and updated as needed.

Foreign key relationships maintain data integrity across tables.

Access Control and Security

- Authentication: Users and admins login through separate roles.
- Authorization: Admins have access to sensitive actions (e.g., updating animal records, approving adoptions).
- Input Validation: Form inputs are validated on both client and server side.
- Password Security: Passwords are stored using hashing (e.g., SHA256 or bcrypt).
- SQL Injection Protection: Use of parameterized queries or ORM to prevent injection attacks.

Global Software Control

Global control flow is managed via ASP.NET MVC routing. Upon each request, the routing engine determines the appropriate controller and action.

- Error Handling: Global error filters and try-catch blocks are used.
- Session Control: Login sessions are stored and validated per request.

Boundary Conditions

The system handles boundary conditions such as:

- Invalid Login: Users receive appropriate error messages.
- Duplicate Adoption Request: System checks if a request for the same animal already exists.
- Deleted Animals: Foreign key rules ensure referential integrity (e.g., ON DELETE SET NULL).
- Empty Fields: Mandatory fields are enforced via validations.

19d. Subsystem Services

Content:

The web-based animal adoption platform is organized into multiple subsystems to manage different responsibilities within the application efficiently. Each subsystem is designed to encapsulate specific services that support modularity, separation of concerns, and scalability. The primary subsystems are:

Authentication and User Management Subsystem

Handles registration, login, logout, and role-based access control.

Services:

- *registerUser()*
- *loginUser()*
- *assignUserRole()*

Animal Records Management Subsystem

Allows authorized personnel (shelter employees) to add, update, or delete animal records, along with details such as health status, neutering status, and breed.

Services:

- *addAnimal()*
- *updateAnimalHealth()*
- *uploadAnimalImage()*

Adoption Request Management Subsystem

Enables users to submit adoption requests and allows administrators to approve or reject them.

Services:

- *submitAdoptionRequest()*
- *reviewRequestByAdmin()*
- *updateAdoptionStatus()*

Content and Article Management Subsystem

Manages blog articles, guides, and breed-related informational content for users.

Services:

- *addArticle()*
- *editArticle()*
- *viewArticleList()*

Admin Panel Subsystem

Offers authorized admin users a dashboard to oversee all operations, including user management, animal data, and adoption processes.

Services:

- listAllUsers()
- modifyAnimalDetails()
- trackAdoptionHistory()

Motivation:

Subsystem separation improves code organization, makes the system easier to understand and test, and allows different developers to work independently on separate concerns. It also supports scalability and future feature extensions with minimal impact on existing code.

Considerations:

Subsystems were grouped based on logical separation of duties and responsibilities. Role-based access, data dependencies, and maintainability were key factors. The modular design also fits well within the ASP.NET MVC framework structure, aligning with controller-based service mappings.

Example:

For example, when a registered user submits an adoption request via the submitAdoptionRequest() method, the system logs the request, and the reviewRequestByAdmin() method from the Admin Panel Subsystem is triggered to either approve or reject it. Upon decision, updateAdoptionStatus() updates the corresponding animal's adoption record.

19e. User Interface

Content

The user interface of our web-based animal adoption platform was designed with a user-friendly and clean structure in mind. It serves as the primary point of interaction between the end-users (adopters and shelter staff) and the system. We adopted a minimalistic and intuitive design approach to ensure that users can easily access the platform's core functionalities: browsing animals, submitting adoption requests, and managing animal records.

Motivation

The main motivation behind our UI design was to ensure that all users, regardless of their technical background, could navigate the system easily. For adopters, the priority was simplicity and accessibility; for administrators, clarity and control. Therefore, we aimed to minimize cognitive load while maximizing efficiency. Our color palette and layout choices reflect calmness, compassion, and trust, aligning with the sensitive nature of pet adoption.

Considerations

- The interface should be responsive and accessible from both desktop and mobile devices.
- Text and button sizes are chosen to ensure readability and clickability.
- Each page was tested to avoid redundant navigation and to maintain a logical flow.
- Users are provided with instant feedback (success/failure alerts) on their actions such as login, form submissions, and request updates.

Example

The following are the primary pages designed in the system, each with a distinct function:

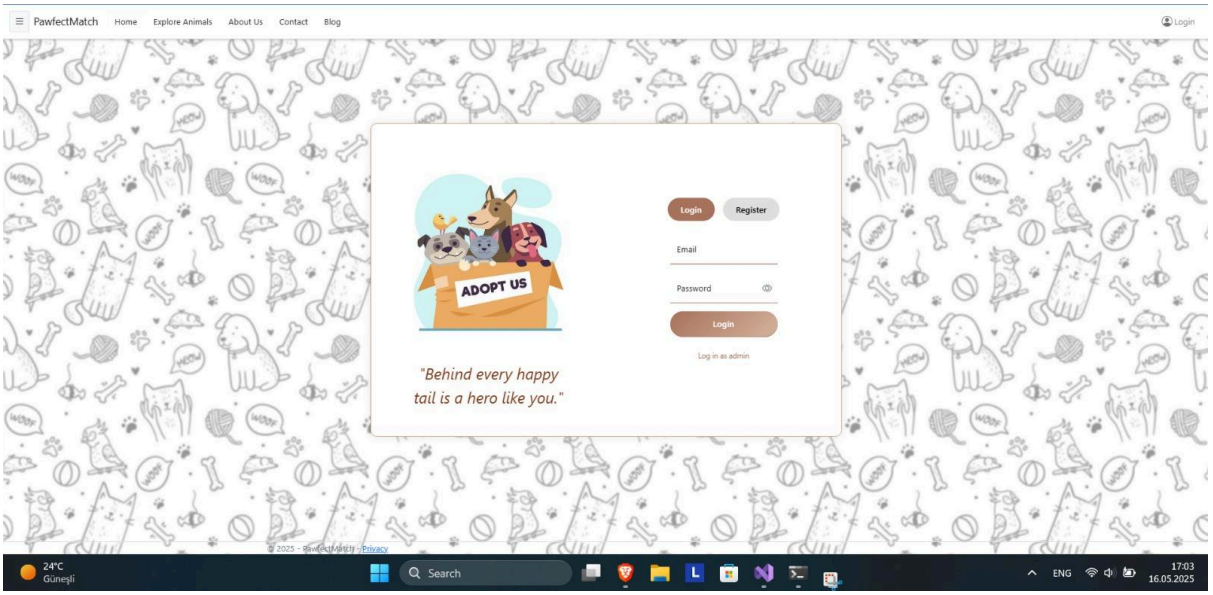

Figure 8. User Login Page

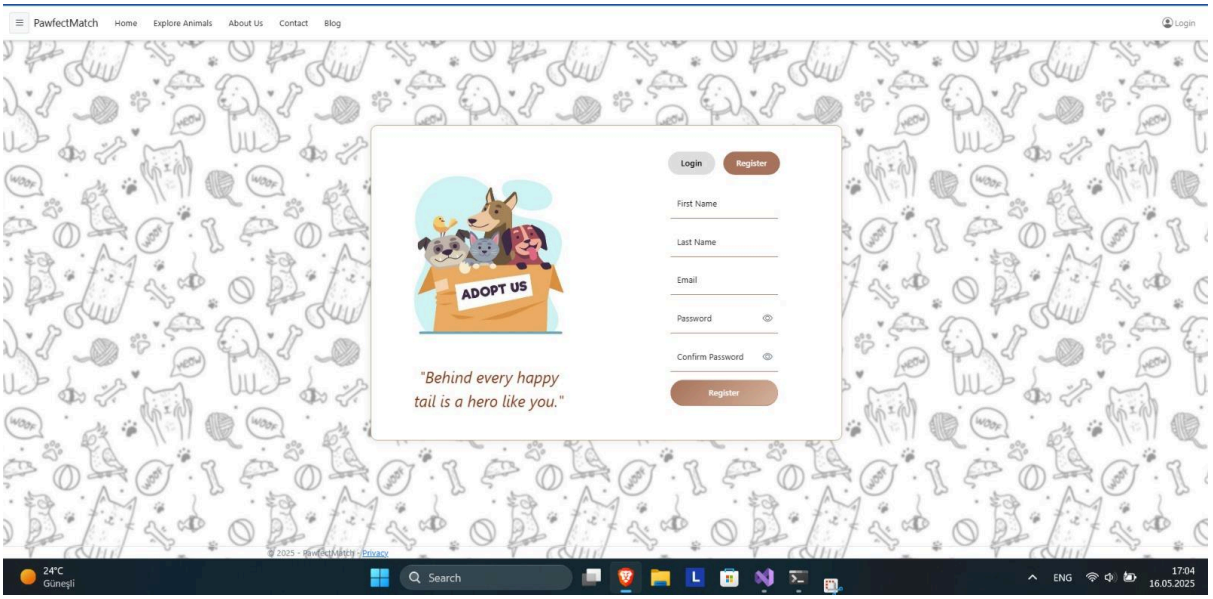

Figure 9. Register Page

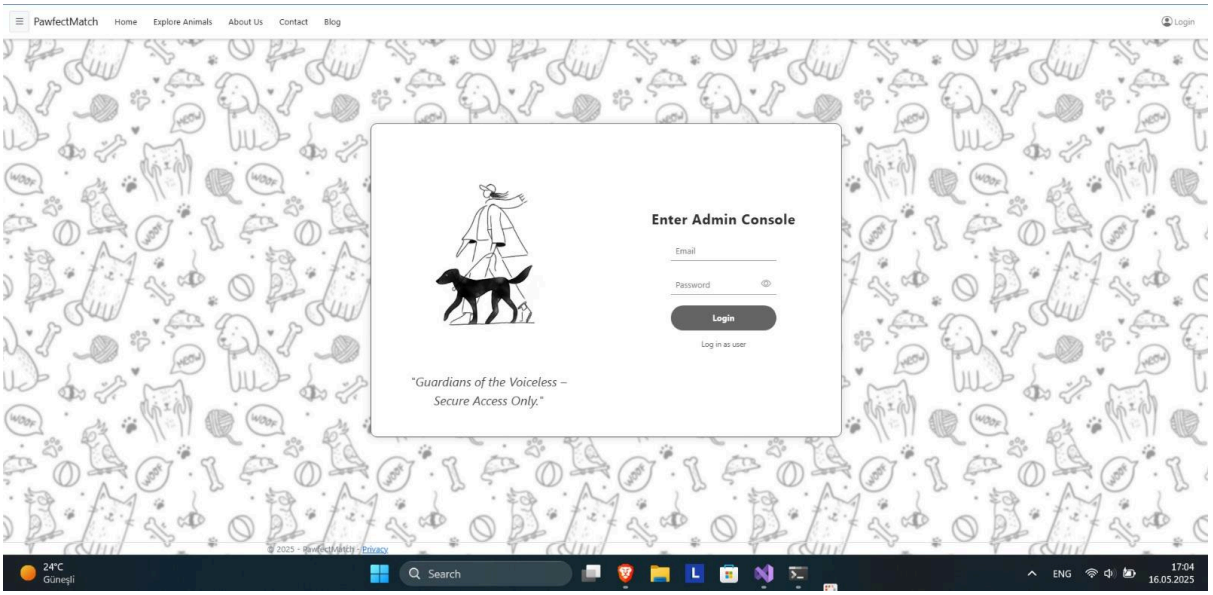

Figure 10. Admin Login

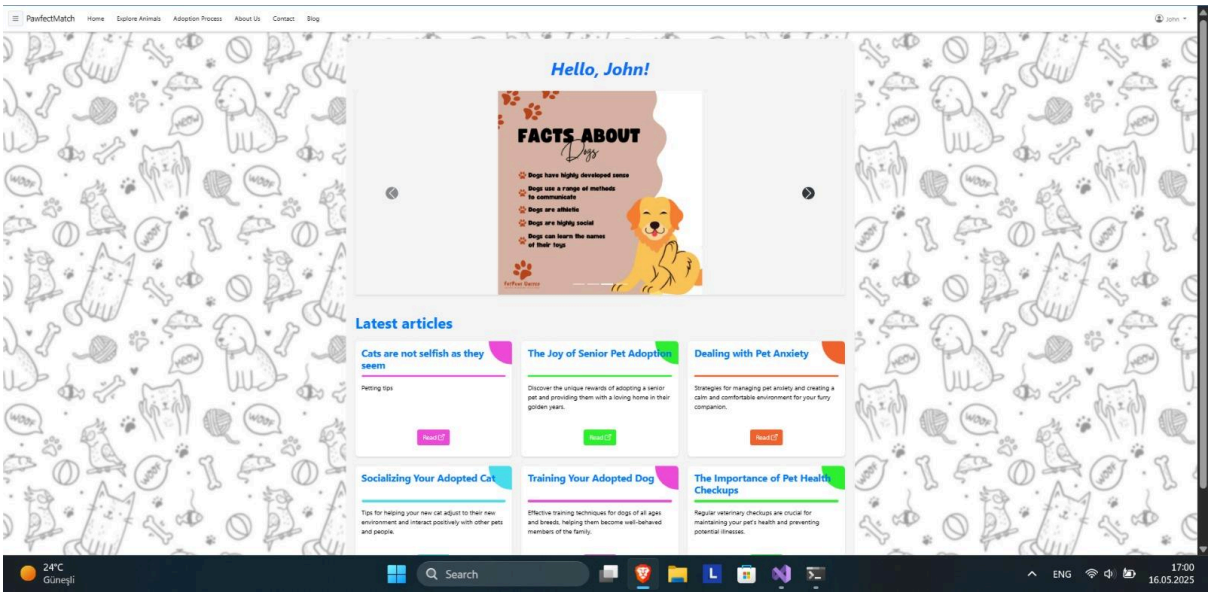

Figure 11. Home Page

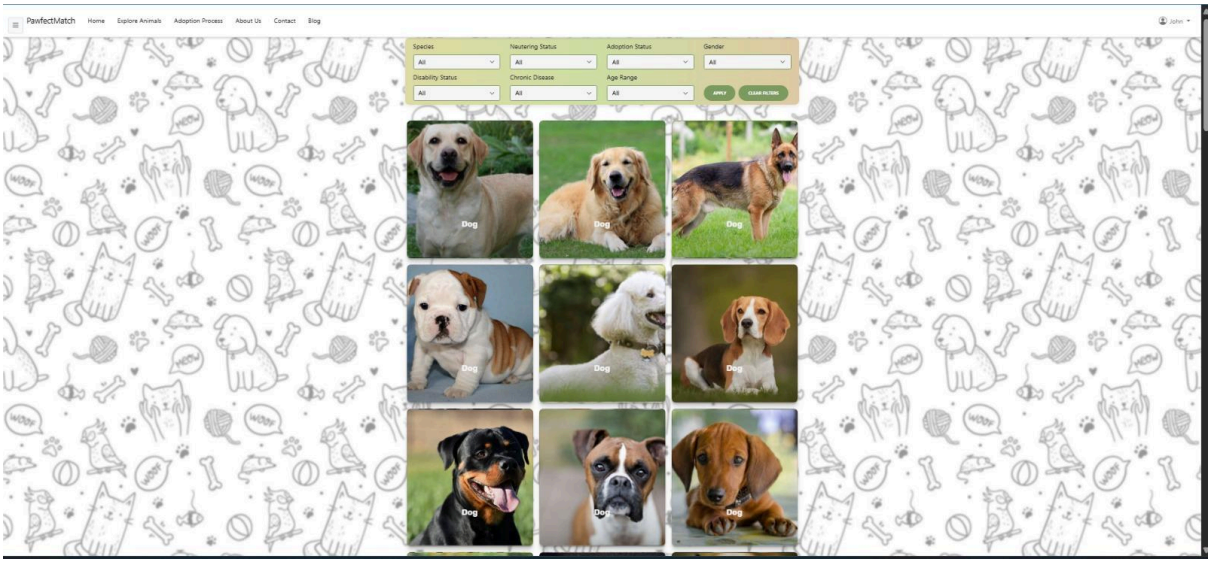

Figure 12. Explore Animals Page

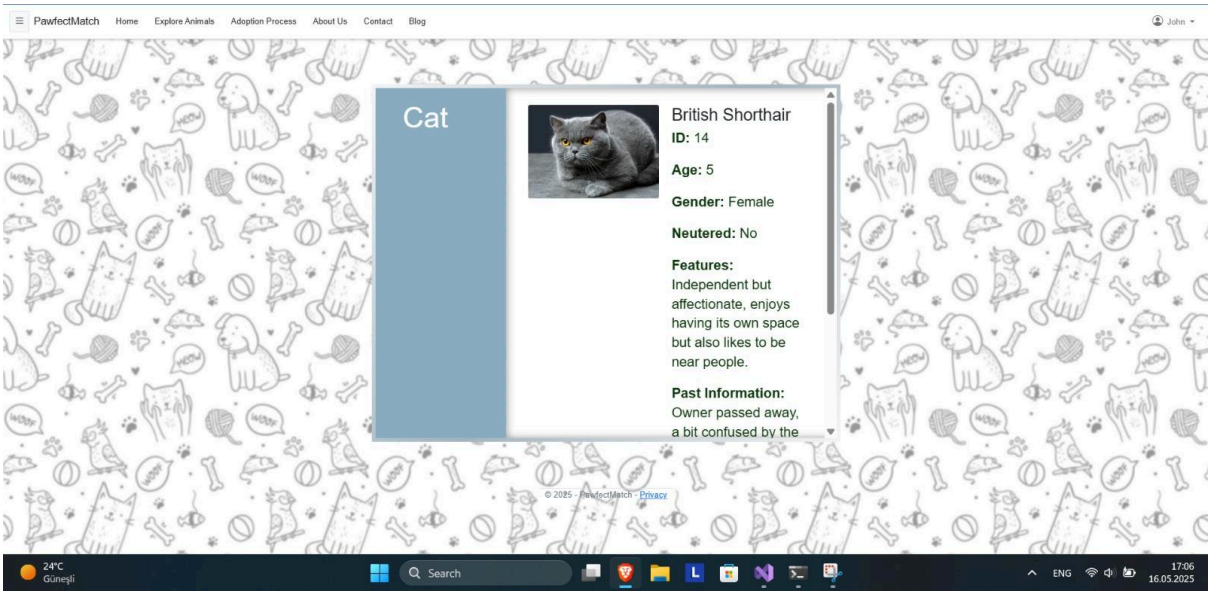

Figure 13. Animal Detail Page

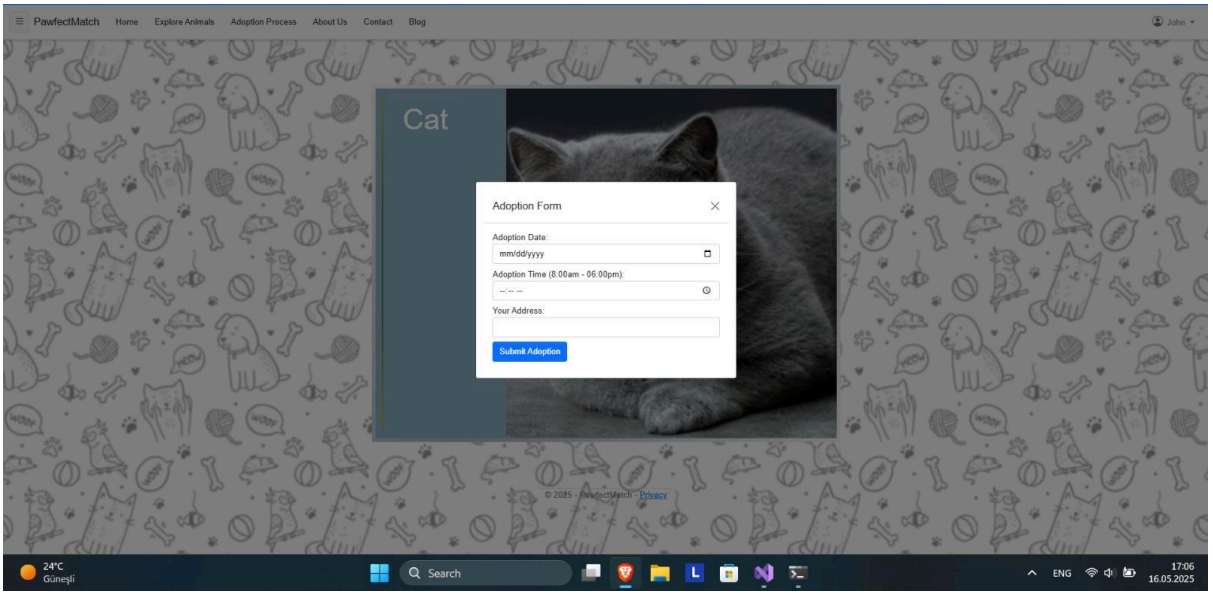

Figure 14. Adoption Form

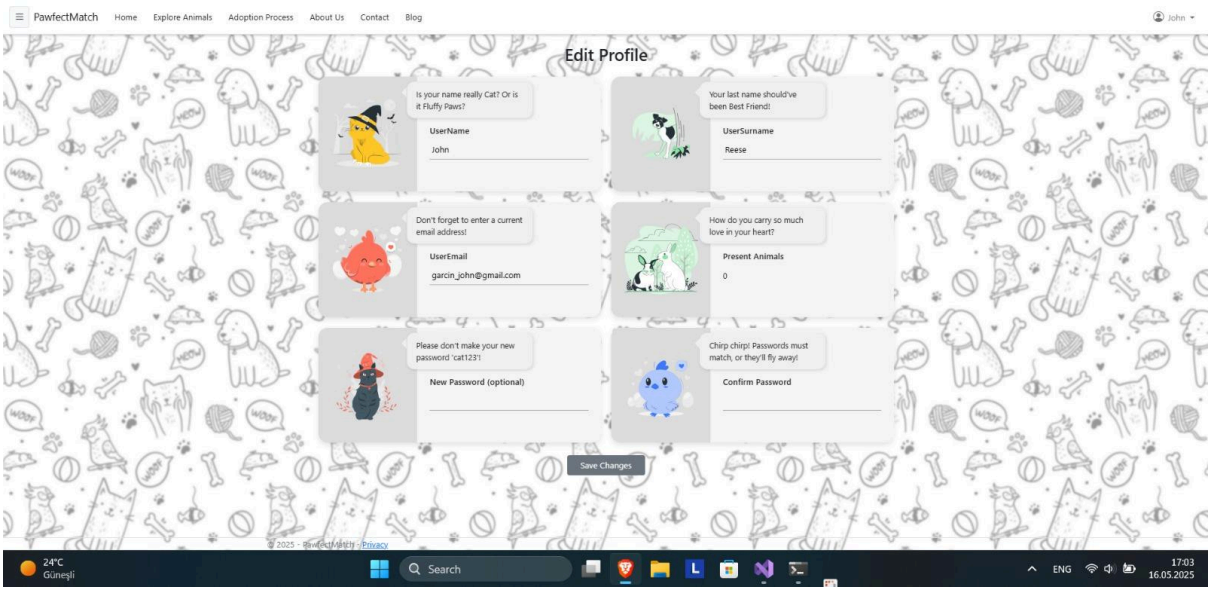

Figure 15. Edit Profile Page

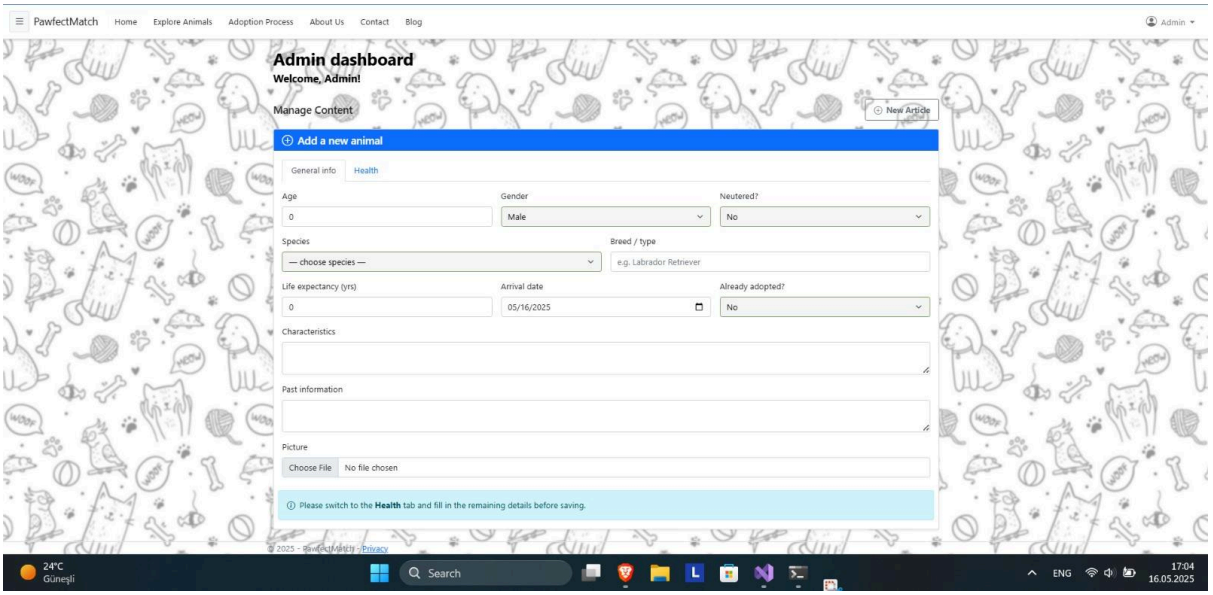

Figure 16. Admin Dashboard Page

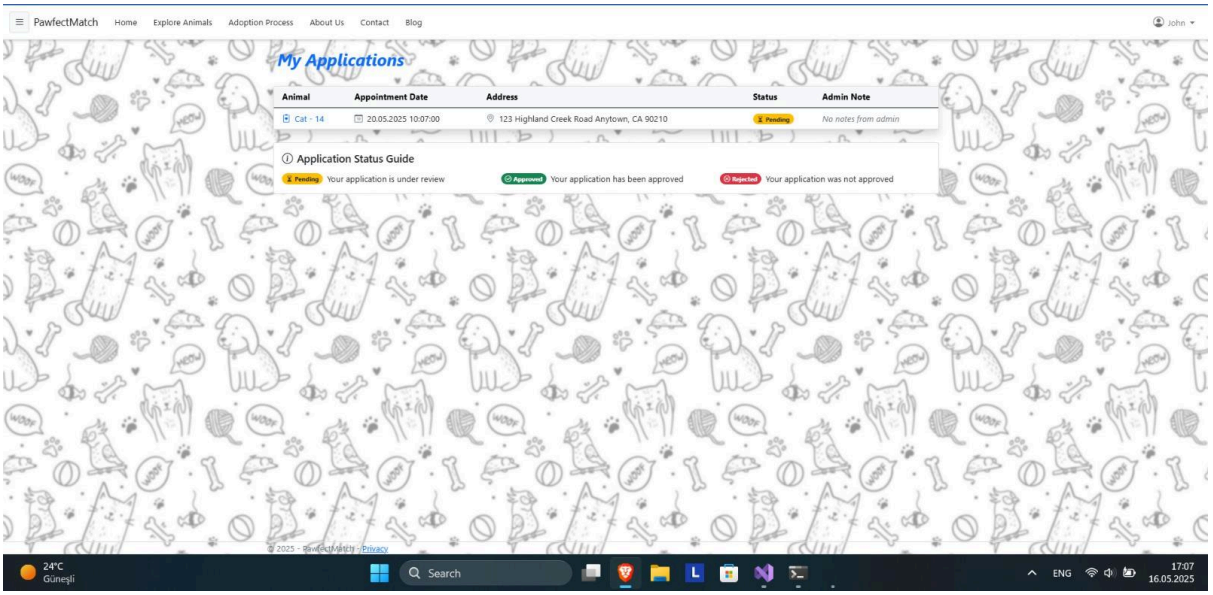

Figure 17. Sending an adoption request as a user

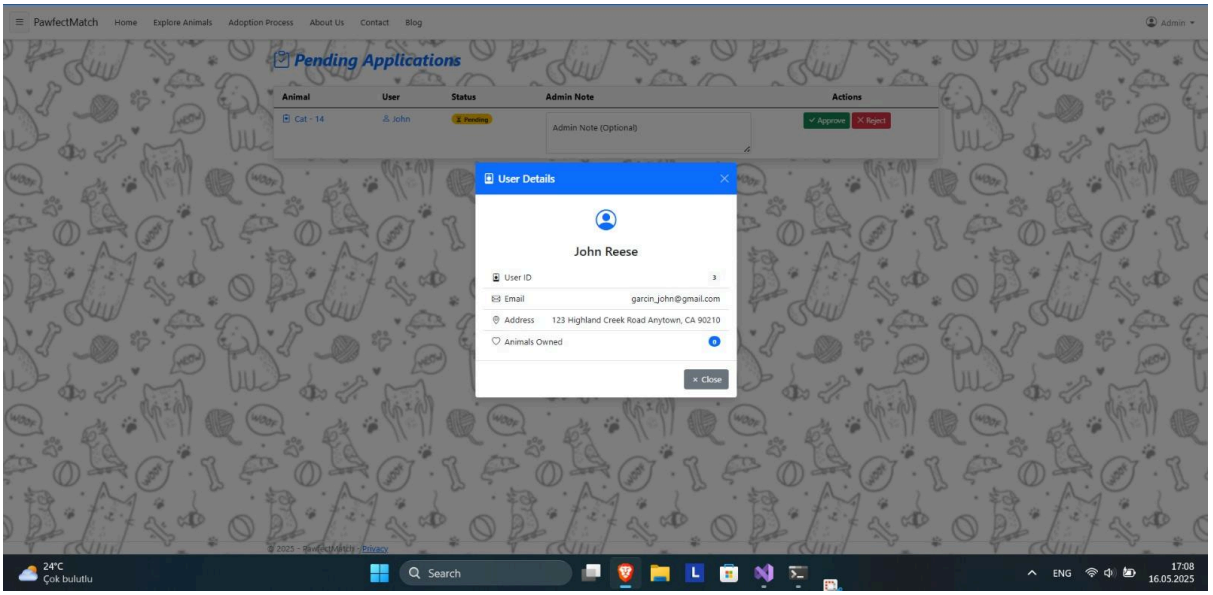

Figure 18. Receiving requests as an admin

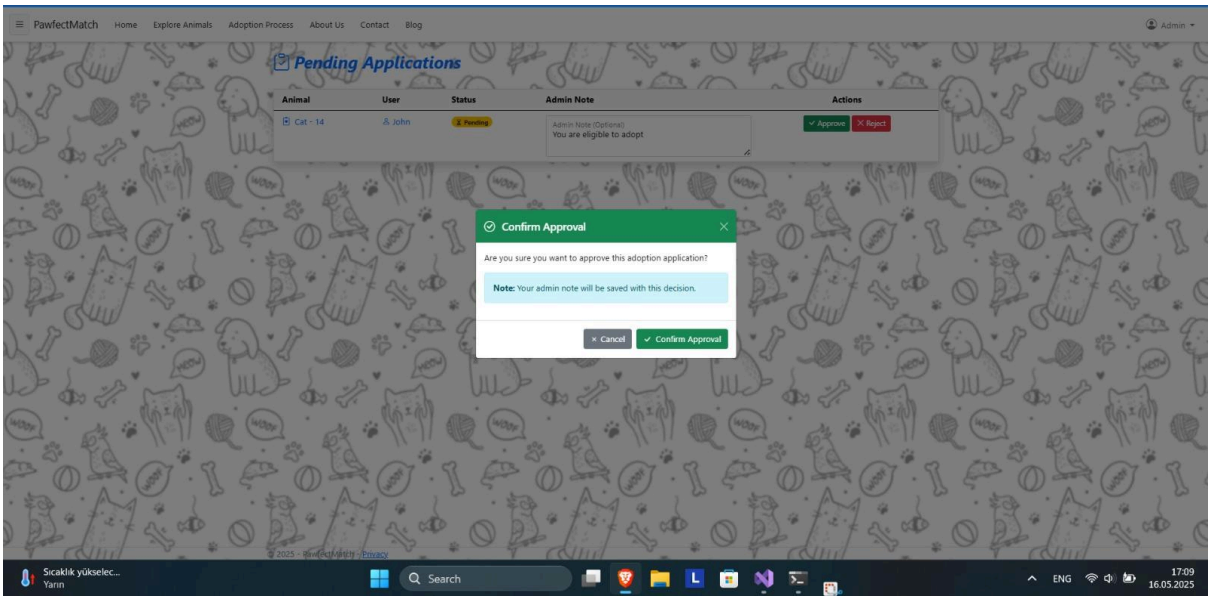

Figure 19. Approval of the request

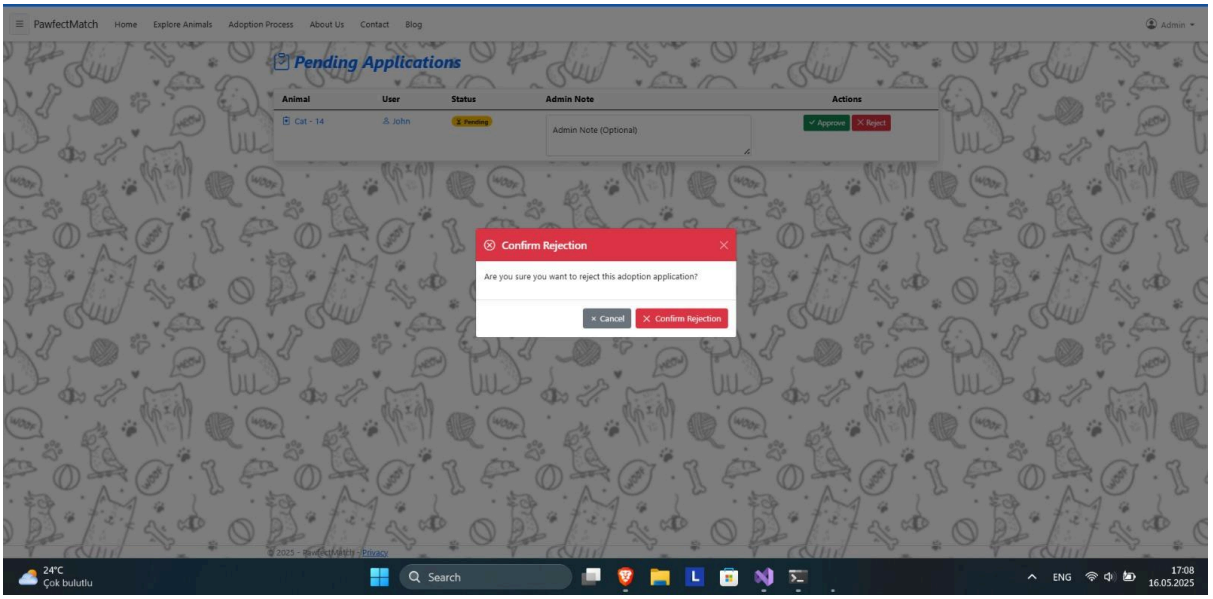

Figure 20. Rejection of the request

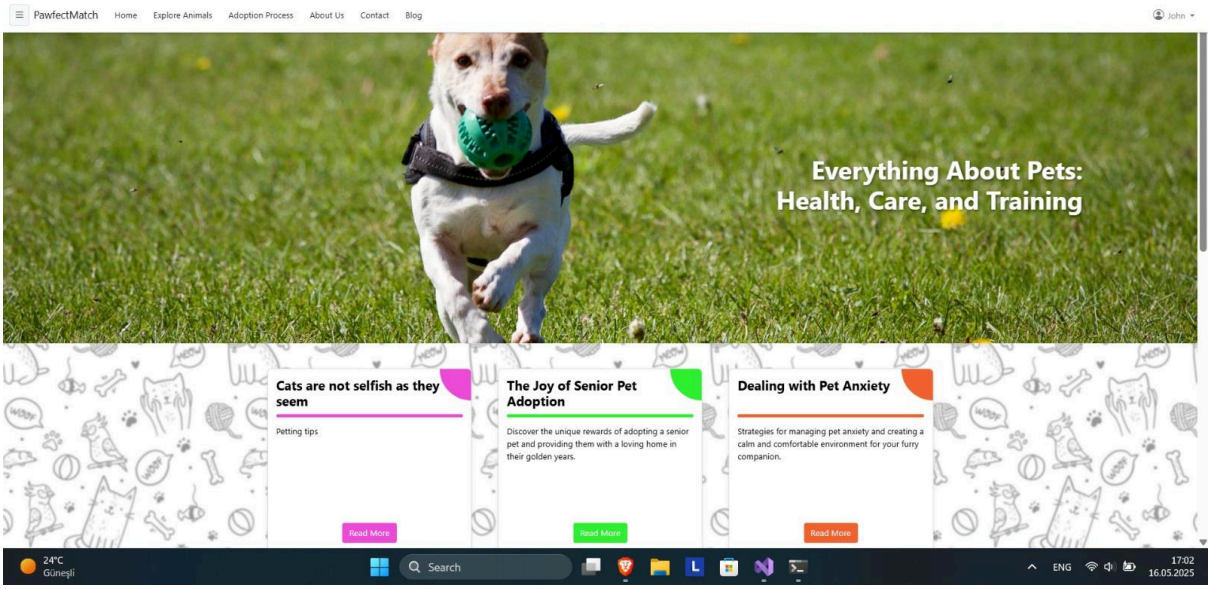

Figure 21. Blog Page

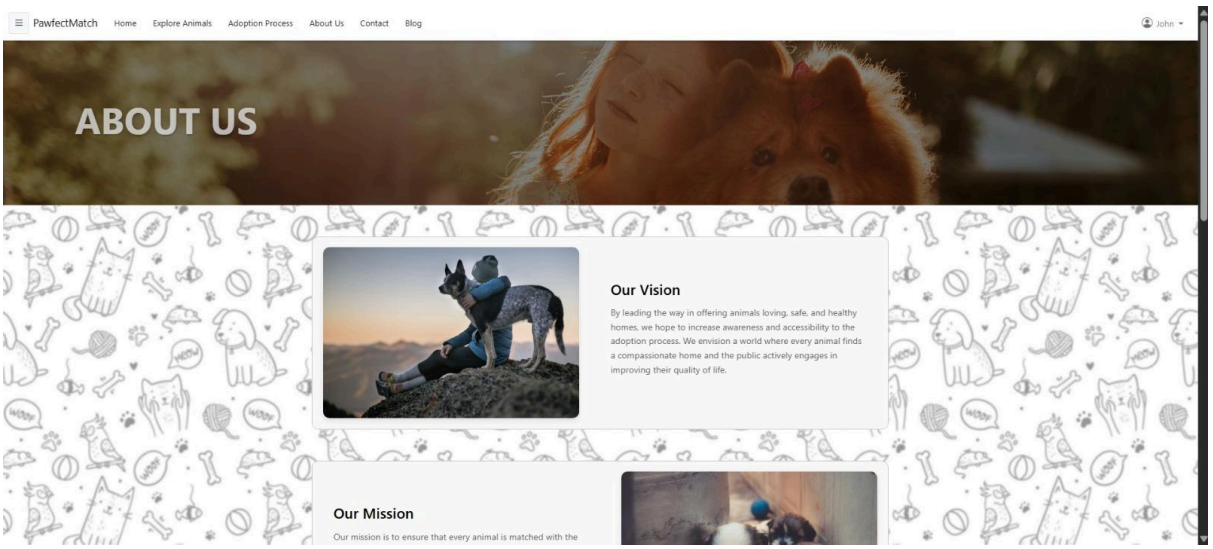

Figure 22. About Us Page

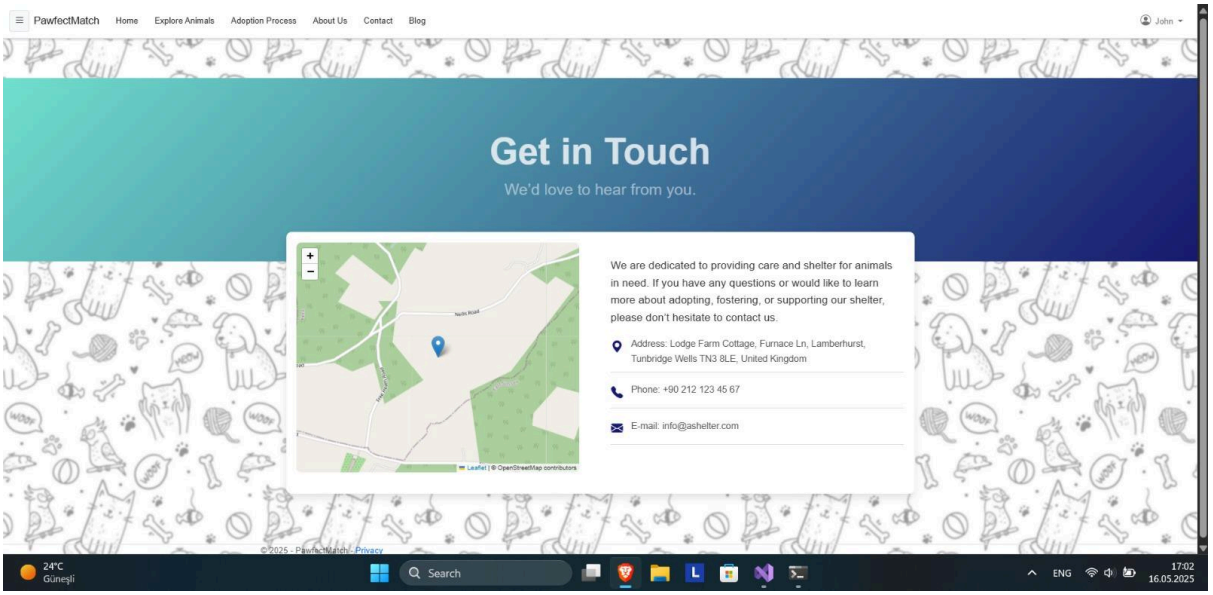

Figure 23. Contact Page

20. Object Design

20a. Object Design Trade-offs

Content:

Our project follows an object-oriented design paradigm using ASP.NET MVC. While designing the objects, we aimed to ensure a balance between cohesion, low coupling, reusability, and simplicity. However, certain trade-offs were necessary to meet project deadlines and team skill levels.

Motivation:

Object design trade-offs were mainly driven by the need to simplify data access and view binding. For example, although implementing separate DTOs (Data Transfer Objects) could improve modularity, we chose to use entity models directly in views to speed up development.

Considerations:

- **Trade-off #1:** Direct use of Entity Framework models in Views instead of creating ViewModels to simplify development.
- **Trade-off #2:** Limiting inheritance to avoid complexity, even though some classes (e.g., Animal types) could benefit from it.
- **Trade-off #3:** Grouping responsibilities inside Controllers rather than fully abstracting services into layers due to project scale.

20b. Interface Documentation Guidelines

Content:

Interfaces were documented following a consistent approach for better collaboration among team members. The method signatures and expected inputs/outputs were briefly described as XML comments in C#.

Motivation:

Documenting interfaces allows new developers to understand the responsibilities of classes and services quickly and ensures smoother future maintenance.

Considerations:

- Every public method includes XML documentation (`///`) explaining its purpose.
- Interfaces follow the `I` naming convention (e.g., IAnimalService).
- We avoided excessive documentation to prevent outdated comments.

20c. Packages

Content:

We organized our project using logical package (namespace) separation within the ASP.NET MVC framework.

Motivation:

Proper packaging allows better organization of code, modularity, and easier team collaboration.

Considerations:

- Packages follow MVC conventions: Models, Controllers, Views, Services, Interfaces.
- Views are grouped per controller in subfolders.

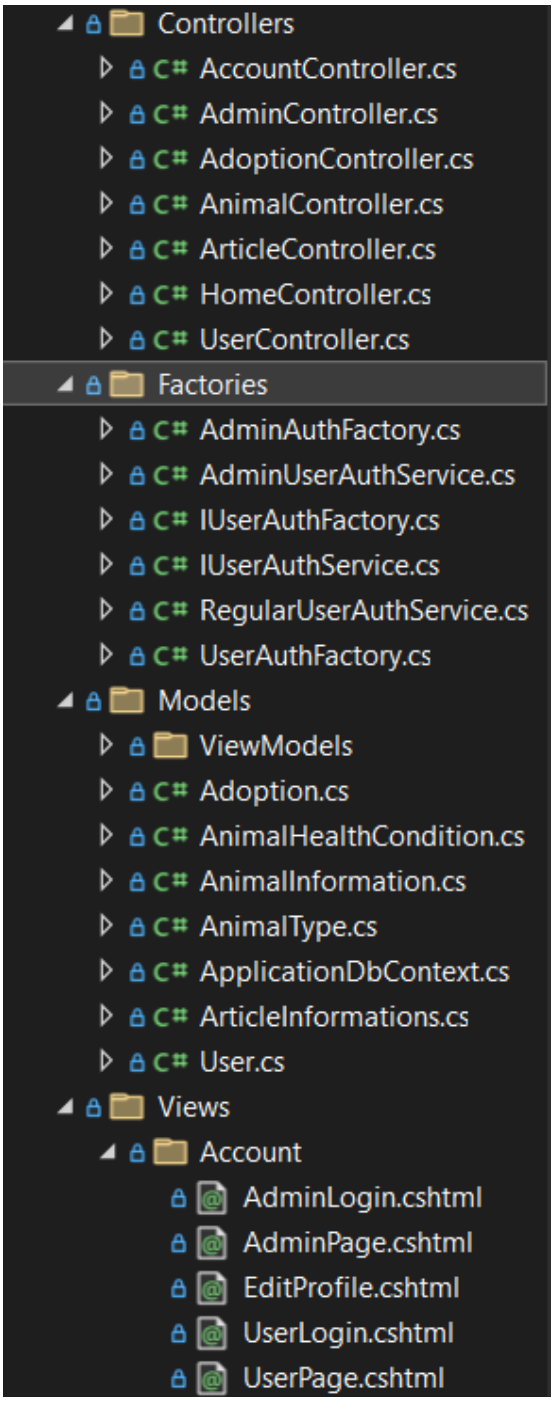

Figure 24. Class structure in the solution explorer

20d. Class Interfaces

Content:

Class interfaces define how different components of the system interact. We used interfaces primarily for services to abstract implementation logic from controllers.

Motivation:

Using interfaces improves testability and maintainability and promotes the use of dependency injection.

Considerations:

- Controllers depend on interfaces, not concrete implementations.
- All services implement a related interface.
- Class interface design follows SOLID principles where applicable.

```
namespace animal_shelter_app.Factories
{
    5 references
    public interface IUserAuthService
    {
        5 references
        User Authenticate(string email, string password);
    }
}
```

Figure 25. An interface example from the project

21. Test Plans

21a. Features to be Tested / Not to be Tested

Content:

This section outlines which features of the system will undergo testing and which ones will be excluded.

Motivation:

To prioritize testing efforts on critical components and ensure essential functionalities are validated before deployment.

Considerations:

- Time and resource limitations may prevent full testing of all features.

- Core functionalities like adoption flow and user login will be prioritized.

Feature	Test Case
User Registration/Login	tested
Admin Login	tested
Edit Profile Information	tested
Animal Listing	tested
Adoption Request	tested
Admin Animal Approval	tested
Admin Animal Rejection	tested
Blog Article View	tested
Blog Article Creation	tested
Adding a new animal	tested

Table 1. Test case status table for project's features

21b. Pass/Fail Criteria

Content:

Defines the conditions under which a test case is considered passed or failed.

Motivation:

To ensure consistent and objective assessment of testing results.

Considerations:

- Expected outputs must match actual outputs.
- User interactions must not cause system crashes or unhandled exceptions.

Example:

- **Pass:** The system displays the animal detail page correctly when clicked from the listing.

- **Fail:** The system throws a server error (500) when submitting the adoption form with valid data.

21c. Approach

Content:
Describes the overall strategy and types of testing that will be performed.

Motivation:
To follow a structured approach and ensure complete coverage of functional and non-functional aspects.

Considerations:

- Testing will be manual, focusing on black-box functional testing.
- Use of dummy data for testing adoption flow and admin controls.

Example:

- **Unit Testing:** Performed on service-level methods (e.g., SubmitAdoptionRequest()).
- **Integration Testing:** Ensuring user registration integrates with email confirmation (if implemented).
- **System Testing:** Simulating full user flow: registration → browse animals → submit adoption.
- **Acceptance Testing:** Done with real users to get feedback on usability.

21d. Suspension and Resumption

Content:
Defines when testing should be paused and how it will be resumed.

Motivation:
To avoid wasting time testing on unstable builds or unresolved critical bugs.

Considerations:

- Testing will be suspended in case of blocker issues (e.g., server crashes).

- Resumed once the issues are resolved and verified by developers.

Example:

- **Suspend:** When animal listing does not load due to database connection failure.
- **Resume:** After the database connectivity issue is fixed and tested independently.

21e. Testing Materials (Hardware / Software Requirements)

Content:

Lists the tools and environments used for testing.

Motivation:

To ensure all testers work under consistent conditions and know what environment is needed.

Considerations:

- The project runs on ASP.NET MVC and MySQL; test environment must replicate this stack.
- Browsers like Chrome, Edge, and Firefox will be used.

Hardware / Software	Requirements
OS	Windows 10,11
Browser	Chrome, Brave
.NET Runtime	.NET 0.8
Database	MySQL Workbench
IDE	Visual Studio 2022
Device	Laptop with minimum 8 GB RAM

Table 2. Hardware/ Software requirements table

Test Cases

Content:
Details the specific scenarios and steps that will be tested.

Motivation:
To ensure consistent, repeatable testing procedures.

Considerations:

- Test cases should cover both normal and edge-case behavior.
- Each test includes the expected result.

21f. Testing Schedule

Content:
Timeline and phases in which testing activities are carried out.

Motivation:
To organize testing efforts and ensure testing aligns with development milestones.

Considerations:

- Testing begins after main modules are complete.
- Final week allocated for user acceptance and bug fixes.

Date	Activity
1-21 March	Unit testing of services Login and animal list testing Edit profile information testing
21 March - 14 April	Create blog articles and view testing Animal filter testing Animal add testing
14 April - 6 May	Send request testing Approval and rejection testing

Table 3. Testing schedules table along with planned test activities

IV. Requirements

22. Product Use Cases

22a. Use Case Diagram

Figure 26. Use case diagram

23. Performance Requirements

23a. Speed & Latency

- All public pages (Home, Explore Animals, Specific Animal) must render "visibly complete" within 1 second on a standard home connection (≈25 Mbps, Istanbul).
- Form posts (login, adoption request) must give the user a response in < 3 seconds.

23b. Precision / Accuracy

- Adoption date-and-time entries must be stored to the minute (seconds and milliseconds are discarded).
- Animal ages are shown as whole years; fractional months are not displayed to keep the UI simple.

23c. Capacity

- The database must comfortably hold 1000 animals and 10000 adoption records without a noticeable slowdown for visitors.

- The image store under `wwwroot/animals` should accommodate 20 GB of photos before the shelter needs to expand disk space or redirect the directory to cloud storage.

24. Dependability Requirements

24a. Availability

- The public website should be reachable 99.5 % of the time in any calendar month (≈ 3½ h downtime max).

- A static “Temporarily unavailable” page is displayed if the database cannot be reached.

24b. Robustness / Fault-Tolerance

- Corrupted or missing animal photos are replaced by a neutral placeholder image; the rest of the page continues to work.

- Duplicate adoption clicks are ignored by checking for an existing *pending* record for the same animal-ID / user-ID pair.

25. Maintainability & Supportability Requirements

25a. Supportability

- Every controller and service class contains concise XML comments so that automated documentation builds without warnings.

25b. Adaptability

- Adding a new animal *species* (e.g., “Ferret”) only requires inserting rows in the tables; no code changes are necessary.

25c. Scalability / Extensibility

- The filtering component is written so that new drop-down criteria (weight, colour, etc.) can be added by appending one extra SQL `WHERE` clause.

25d. Longevity

- The project targets .NET 8 LTS, supported by Microsoft until November 2026, ensuring three-year security coverage and can be extended further with simple package upgrades on NuGetPackages.

26. Security Requirements

26a. Access

- The system recognises two roles only: Admin (a single, pre-created account) and User.

- All admin URLs are grouped under /Admin/* and protected by a simple role check.

26b. Integrity

- Every form that changes data carries an anti-forgery token.

- Images are accepted only if the file extension is .png or .jpg and the MIME type matches.

26c. Privacy

- User passwords are hashed (SHA-256 + salt).

- Adoption addresses are visible solely on the Admin page and never sent in e-mails.

- E-mail format is currently checked with a simple “contains @ and dot” rule; a stricter RFC-compliant check is noted as future work.

27. Usability & Human-Factors Requirements

27a. Ease of Use

- A first-time visitor can reach the “Adopt Me” form in three clicks or fewer starting from the home page.

27b. Personalisation & Internationalisation

- The interface is delivered in English by default and can be further extended to support multiple languages.

27c. Understandability & Politeness

- All error messages use plain, friendly language (“Please choose an image under 2 MB”) and never expose stack traces.

28. Look-and-Feel Requirements

- The site uses a calm green palette (#7C9C62) consistent with the shelter’s general theme.

- Buttons have softly rounded corners; cards lift slightly on hover to convey interactivity.

29. Operational & Environmental Requirements

29a. Expected Physical Environment

- The web server will run on a small virtual machine in the shelter’s office (Ubuntu Linux, SSD). No special hardware is required on site. MySQL already handles the local server side, a working laptop to access the webpage is enough.

29b. Productization

- The application will be distributed as a website link for everyone to access and login.

30. Cultural & Political Requirements

- Only rescue and adoption content is allowed. Breeding, sales, or political advertisements are strictly disallowed.

31. Legal Requirements

31a. Compliance

- The shelter must be able to delete a user's personal data on request (GDPR / KVKK "right to be forgotten").

- All photos are either taken by staff or explicitly licensed; a photo-source field is stored but may be left blank if staff-owned.